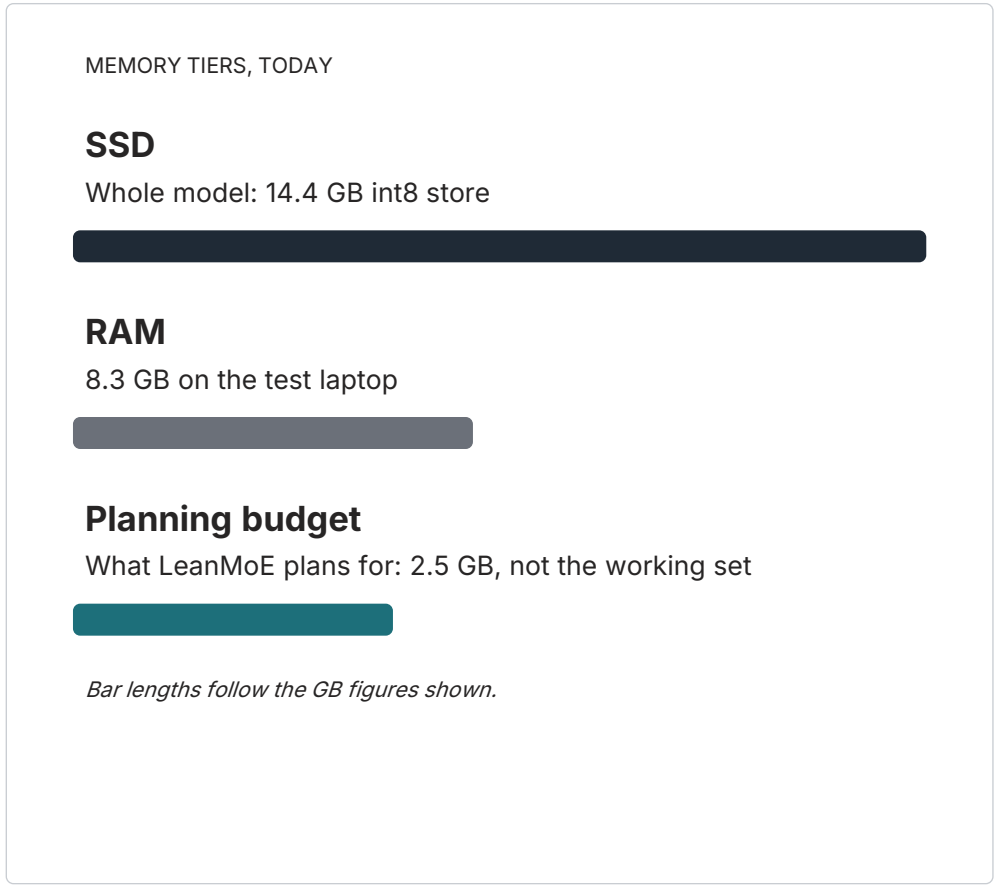
Large AI.

Smaller memory footprint.

Memory-aware inference of MoE LLMs on a Windows laptop.

Abdullah Ali Alanazi

HUMAIN Challenge · Deliver AI with less compute



02 · THE PROBLEM

The model fits. *RAM doesn't.*

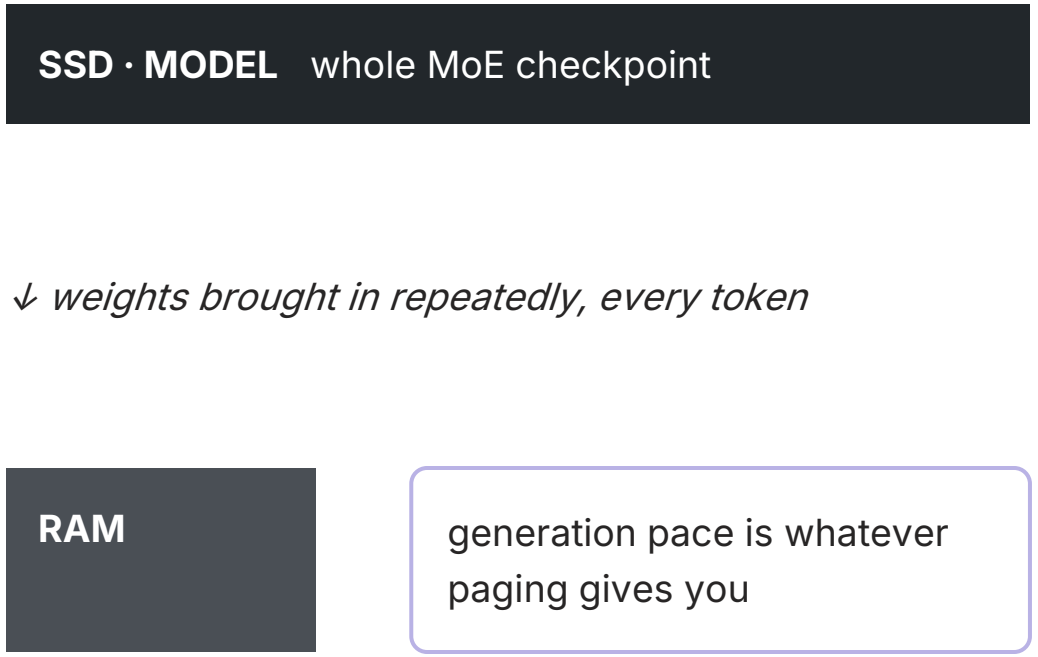
Large MoE models load fine onto a modern SSD.

Their expert weights, however, cannot all stay resident in RAM.

During generation, weights must be brought in repeatedly.

Test laptop: 8.30 GB RAM against a 14.4 GB model store.

WHAT ACTUALLY HAPPENS



03 · THE SOLUTION

Keep the experts you need.
Fetch the rest on demand.

1 Store on SSD

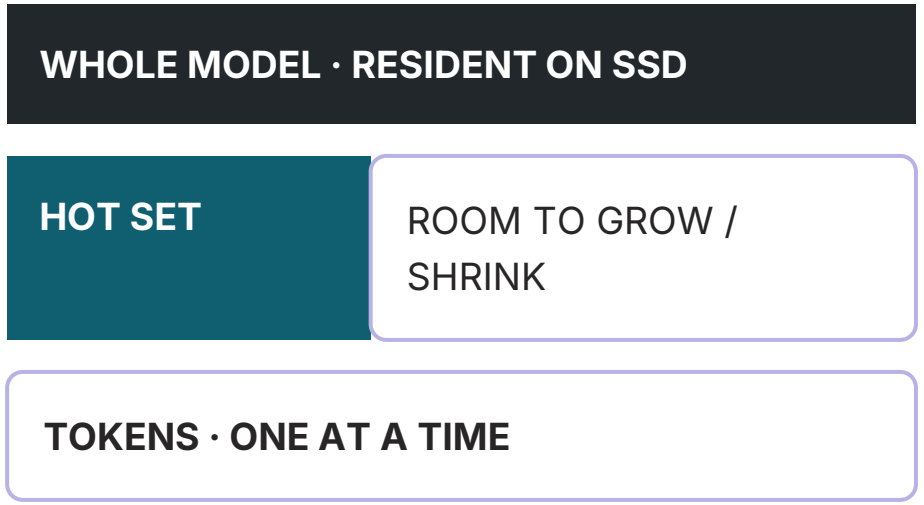
The full MoE checkpoint lives on disk (14.4 GB int8 store). Cold storage, no surprise.

2 Cache a *selected* subset in RAM

Caching keeps recently used experts hot · **prefetching** anticipates the next call · **resizing** shrinks the working set when memory pressure rises.

3 Generate tokens with Qwen

Prefetching aims to have the next expert ready; when it is not, the layer waits for the disk.



Qwen supplies the model and the answers · **LeanMoE** manages **expert-weight access**.

Shown on a laptop.
Tested on a VM.

DEMONSTRATED

Developers and small teams running MoE *locally* on a memory-constrained Windows laptop.

One laptop: Core i5-12450H, 8.30 GB RAM, NVMe SSD

One model: Qwen1.5-MoE-A2.7B-Chat, int8

Single-machine workflow, no cluster

TESTED

One shared Windows VM was tested (Azure D4ds_v4, 16 GB). Automatic cache sizing helped LeanMoE, but Windows paging stayed faster (slide 8).

Broader server or data-center benefits remain unverified.

05 · LAPTOP PERFORMANCE

2.07× on average.

Not every run.

2.07×

ratio of mean decode rates: 1.842 ÷ 0.891 tokens/s

MEAN DECODE, TOKENS/S

LeanMoE	1.842
Windows paging	0.891

PER-RUN RATIO: LEANMOE ÷ PAGING

fact · run 1	2.92×
fact · run 2	0.92×
code · run 1	3.07×
code · run 2	1.92×
explain · run 1	11.2×
explain · run 2	1.95×

Grey bar = paging was faster (below 1×).

SCOPE

LeanMoE faster in 5 of 6 matched runs.

Pairs range 0.92× to 11.2×; median 2.44×.

2.07× is the ratio of mean rates, not a mean of ratios.

Decode only: loading and prefill excluded.

Windows file cache was never cleared.

06 · LIVE ADAPTATION

Mid-run, the plan shrinks.

After token 20, the working set is cut. The generation keeps going.

PLANNING BUDGET

3.5 GB → **2.5 GB** AFTER TOKEN 20

CACHED EXPERTS

124 → **59** WORKING SET TRIMMED

PROCESS PRIVATE MEMORY (COMMITTED)

≈ 2.39 GB after the cut, generation still running (uncut reference: 2.83 GB, 109 experts).

Budget is a planning target, not a hard limit. Source: results/EVIDENCE.md, primary pair.

OUTPUT TOKEN IDS MATCHED

80 / 80

exactly the same token IDs as the uncut reference.

One prompt, one configuration. Matching tokens show execution consistency, not answer correctness.

07 · TASK CORRECTNESS

Eight of twelve answered *correctly.*

Same 8 of 12 in each mode, for first and warm requests, within the 120-second post-load limit.

01 Addition **Correct**	**02** Multiplication **Wrong**	**03** Multi-step arithmetic **Wrong**	**04** Linear equation **Correct**
05 Quadratic equation **Correct**	**06** Harder quadratic **Wrong**	**07** Capital city **Correct**	**08** Chemical symbol **Correct**
09 Moon landing year **Correct**	**10** Marbles passage **Wrong**	**11** Train timetable **Correct**	**12** Oldest in list **Correct**

TWELVE TASKS

8 / 12

Wrong: 02 multiplication, 03 multi-step arithmetic, 06 harder quadratic, 10 marbles passage.

Median request latency: LeanMoE 14.9 s first, 14.1 s warm. Windows paging 38.0 s, 46.0 s.

Small study: one model, one laptop. Not general model quality; matching tokens do not show correctness.

08 · CLOUD COMPARISON

One 16 GB Windows VM.

Paging was a bit faster.

DECODE THROUGHPUT, TOKENS/S



AUTOMATIC VS SMALL CACHE

6.58×

Automatic LeanMoE ÷ small-cache LeanMoE decode rate (1.998 ÷ 0.304 tokens/s).

Windows paging stayed faster: 2.198 vs 1.998 tokens/s, about 10% above automatic LeanMoE.

Unequal memory, not an equal-memory test: sampled peak working set 2.9 GB (small cache), 12.7 GB (automatic), 13.3 GB (paging). Windows managed its own file cache, which was not controlled.

One prompt, two main repeats.

09 · OWNERSHIP & CONTRIBUTION

What I did in the hackathon.

What I did before.

EARLIER · BY ME · EXPERTRELAY

The engine and its pre-Windows pieces.

ExpertRelay engine: int8 store format and readers
fused NumPy runtime
expert cache
predictor (prefetch)
LiveBudget
startup guard

LEANMOE · THIS SUBMISSION

What the Windows-path submission adds on top.

Windows port and integration
corrected memory accounting
evaluations: laptop, cloud VM, 12-task check
dashboard and streaming
automatic cache sizing
budget-cut check against an uncut run

Model: Qwen supplies the model and answers; this submission does not retrain or modify it. · Boundary recorded in **DECISIONS.md**

Try *LeanMoE.*

01 · CLOUD DEMO

No visitor login. *Just open and run.*

https://leanmoe-demo-1.eastus.cloudapp.azure.com/

Inference runs on the server. Visitors need no login.

Recorded VM prefill: about 43 s.

02 · SOURCE & EVIDENCE

GitHub, with the *decision record & evidence.*

https://github.com/No6Name8/LeanMoE

DECISIONS.md · results/EVIDENCE.md · docs/TASK_BENCHMARK.md

03 · LOCAL DEMO VIDEO

Recorded on the same *memory-constrained laptop.*

Watch the local demo on Loom

Recorded walkthrough of the local dashboard.